

UE: Parallel Architectures

- AA: Parallel Computing Lab Work
CREMER Samuel

Matrix Multiplication Report

Master in industrial Engineering Sciences Specialization: Life Data Technologies

Table des matières

Introduction.....	4
Description of Equipment used	4
Benchmarks	5
Task 1: Compute a matrix sequentially	5
Value table under Debian Linux	5
Value table under Windows	5
Graphical Representation of Data	5
Commentary	6
Task 1.1: Compute a matrix sequentially with the first optimization	7
Value table under Debian Linux	7
Value table under Windows	7
Graphical Representation of Data	7
Commentary	8
Task 1.2: Compute a matrix sequentially with the second optimization	9
Value table under Debian Linux	9
Value table under Windows	9
Graphical Representation of Data	9
Commentary	10
Task 1.3: Compute a matrix sequentially with the three optimizations	11
Value table under Debian Linux	11
Value table under Windows	11
Graphical Representation of Data	11
Commentary	12
Task 2: Compute a matrix case by case with thread	13
Value table under Debian Linux	13
Value table under Windows	13
Commentary	13
Task 3: Compute a matrix row by row with thread	14
Value table under Debian Linux	14
Value table under Windows	14
Graphical Representation of Data	14
Commentary	15

Task 4: Dynamic number of the threads	16
Value table under Debian Linux	16
Value table under Windows	17
Graphical Representation of Data	17
Commentary	19
Task 4.1: second optimization	20
Value table under Debian Linux	20
Value table under Windows	21
Graphical Representation of Data	21
Commentary	23
Task 4.2: second optimization	24
Value table under Debian Linux	24
Value table under Windows	25
Graphical Representation of Data	25
Commentary	27
Task 4.3: optimisation.....	28
Value table under Debian Linux	28
Value table under Windows	29
Graphical Representation of Data	29
Commentary	31
Task 5: Cuda on GPU	32
Value table under debian linux.....	32
Value table under Windows 11.....	33
Graphical Representation of Data	34
Commentary	36
Task 5.1: CUDA on GPU with the Transpose Optimization.....	37
Value table.....	37
Graphical Representation of Data	38
Commentary	40
General Conclusion	41

Introduction

As part of the Parallel Architectures course, this report explores different approaches for performing matrix multiplication in C++ on both CPU and GPU. The objective is to progressively move from a simple sequential implementation to increasingly parallel and optimized solutions, concretely measuring the impact of each technique on performance and resource utilization.

The implemented methods include sequential computation, parallelization by thread per element, per row, dynamic thread allocation, and finally GPU acceleration via CUDA. Three successive optimizations were also applied: matrix transposition to reduce cache misses, float conversion to exploit AVX/FMAD instructions, and 64-byte memory alignment.

All tests were performed on square matrices ranging from 100×100 to 2000×2000, on the same physical machine running both Debian Linux and Windows 11, allowing for a direct cross-platform comparison.

Description of Equipment used

For this project, I was using a tower PC who has the following characteristics:

- ✓ Operating System 1: Debian GNU/Linux 12 (bookworm) x86_64
- ✓ Operating System 2: Windows 11
- ✓ Kernel: 6.1.0-43-amd64
- ✓ Processor: 11th Gen Intel i7-11700KF (16) @ 4.900GHz
- ✓ Memory (Ram): 32 Go
- ✓ GPU: NVIDIA GeForce RTX 3070 Lite Hash Rate

I also used Gcc 12.2.0

Here are the different tasks that have been requested:

- ✓ Task 1 consisted of calculating a matrix sequentially
- ✓ Task 2 consisted of calculating a matrix by launching one thread per case
- ✓ Task 3 consisted of calculating a matrix by launching one thread per row
- ✓ Task 4 consisted of calculating a matrix by using a dynamic number of threads
- ✓ Task 5 consisted of using CUDA library for GPU calculation

Benchmarks

All benchmarks presented here result from a specific use of threading.

Task 1: Compute a matrix sequentially

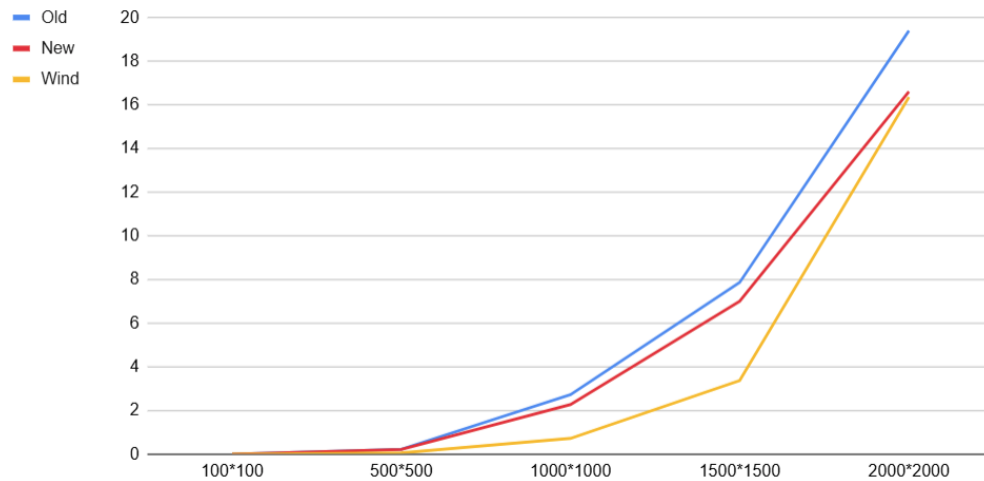
Value table under Debian Linux

Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.00442695	100.61	3.5 Mo
500*500	0.22174	100.01	6.1 Mo
1000*1000	2.27607	100	15
1500*1500	7.00924	100	29
2000*2000	16.6021	100	49

Value table under Windows

New Version - Code (Windows)			
Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.0004816	0	4.5
500*500	0.0637724	73.5	7.3
1000*1000	0.725574	55.99	16
1500*1500	3.37234	70.89	30
2000*2000	16.3514	83.04	50

Graphical Representation of Data



Commentary

This exercise clearly highlights the limitations of sequential matrix computation. As I increase the matrix size, execution time grows rapidly, which is entirely consistent with the theoretical complexity of $O(N^3)$ for matrix multiplication every time I double N , execution time is multiplied by approximately 8, which my measurements confirm.

Comparing my two test environments, I observe that Linux displays higher execution times than Windows across all matrix sizes. For a 2000×2000 matrix, I obtain 16.02s on Linux versus 16.35s on Windows. The values are very close at large scale but Windows shows a much more significant advantage on smaller matrices (100×100 : 0.0044s versus 0.00048s). Based on my observations, these differences are primarily explained by compiler behavior, memory allocator differences, and the runtime environment specific to each OS, rather than any algorithmic advantage.

In both environments, CPU usage approaches 100% throughout the computation, confirming that this workload is entirely CPU-bound. Memory usage, on the other hand, remains modest and grows predictably as N^2 , which makes sense since I am storing three $N \times N$ matrices in memory.

These results convince me of the value of parallelization, which I will explore in the following tasks. Indeed, with a single thread, I am only saturating one core of my CPU while leaving the other 15 completely idle. With my CPU offering 16 logical threads, the performance gain from multithreading will be far from negligible.

Task 1.1: Compute a matrix sequentially with the first optimization

Improvement: Create an additional matrix BT, which is the transpose of matrix B. Use BT instead of B in the computation so that it becomes a multiplication of the rows of matrix A with the rows (rather than the columns) of matrix BT.

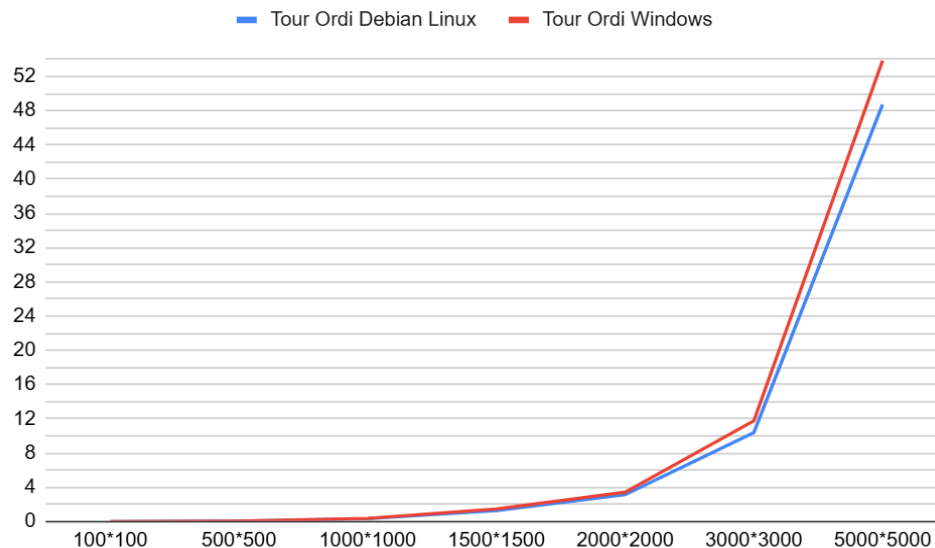
Value table under Debian Linux

Tour Ordi (linux) - B Transposé			
Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.0023264	100.56	3.2
500*500	0.0481845	98.5	6.8
1000*1000	0.346846	100	18
1500*1500	1.27882	100	37
2000*2000	3.14029	100	64
3000*3000	10.4064	100	140
5000*5000	48.7493	100	380

Value table under Windows

Tour Ordi (Windows) - B transposé			
Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.0003893	0	4.9
500*500	0.0496404	62.95	8.2
1000*1000	0.374209	91.86	20
1500*1500	1.48934	61.9	39
2000*2000	3.43022	84.72	65
3000*3000	11.7711	84.02	140
5000*5000	53.8592	85.41	390

Graphical Representation of Data



Commentary

This exercise explores the impact of the first optimization applied to sequential matrix computation: the use of a transposed matrix BT instead of B during the calculation. By doing so, both matrix A and matrix BT are accessed row by row, which eliminates the column-jumping memory access pattern that caused numerous cache misses in the naive version.

Comparing my two environments, I observe that Windows achieves better performance than Linux for all matrix sizes. For a 2000×2000 matrix, I obtain 3.14s on Linux versus 3.43s on Windows, a relatively small difference at large scale. However, for smaller matrices such as 100×100, the gap is more pronounced (0.0023s on Linux versus 0.00039s on Windows), which I attribute to differences in compiler behavior and memory management between the two operating systems rather than any algorithmic difference.

Compared to the naive sequential version, this optimization delivers a notable improvement, particularly as matrix size increases, where cache efficiency becomes the dominant factor. For a 2000×2000 matrix, my Linux version drops from 16.02s (naive) to 3.14s (optimized), representing a speedup of approximately 5x and confirming that memory access patterns have a major impact on performance even in a single-threaded context.

CPU usage remains at approximately 100% in both environments, confirming that the workload remains entirely CPU-bound. Memory usage increases slightly compared to the naive version due to the additional BT matrix, and scales as expected with N^2 .

These results reinforce my motivation to combine this optimization with multithreading in the following tasks, where I expect the sequential memory access pattern of BT to amplify the benefits of parallel execution across my 16 logical threads.

Task 1.2: Compute a matrix sequentially with the second optimization

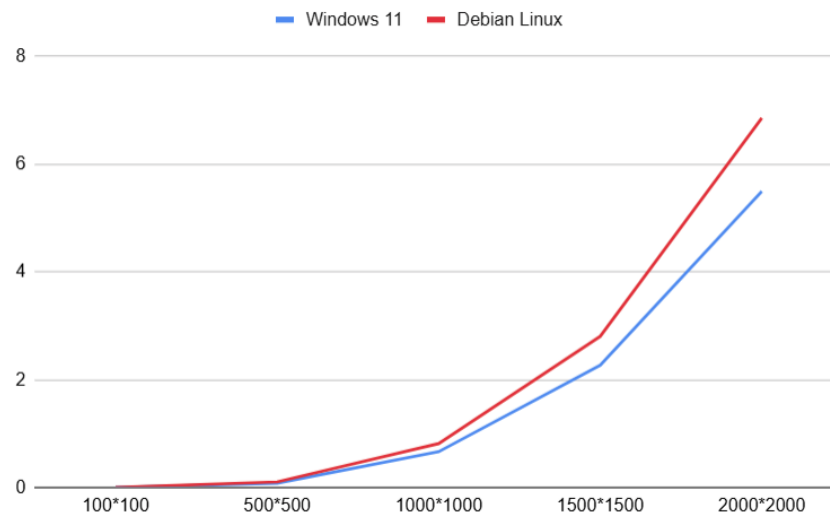
Value table under Debian Linux

Tour Ordi (Debian Linux)			
Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.00122027	101.86	3.2
500*500	0.104083	100	6.8
1000*1000	0.814959	99.99	18
1500*1500	2.80614	100	37
2000*2000	6.8566	100	64

Value table under Windows

Tour Ordi (windows)			
Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.0004576	0	4.9
500*500	0.0971305	80.43	8.2
1000*1000	0.855129	95.01	20
1500*1500	2.93055	88.51	39
2000*2000	7.03664	95.26	65

Graphical Representation of Data



Commentary

In this task, I applied the second optimization to the sequential matrix computation: I converted the data from integers to floats in order to try to take advantage of the FMAD instruction available in the AVX units of my processor, while keeping the transposed matrix BT from the previous optimization.

Comparing my two environments, I notice that Windows remains faster than Linux for all matrix sizes. For a 2000×2000 matrix, I get 6.85s on Linux versus 7.03s on Windows. What surprised me is that for smaller matrices, the gap is much more pronounced, 0.00046s on Windows versus 0.00122s on Linux for a 100×100 matrix. I think this comes from the way each OS handles AVX instructions and compiler optimizations.

But what truly caught me off guard is that this optimization is significantly slower than the first one. On Linux, for a 2000×2000 matrix, I went from 3.14s to 6.85s, almost twice as slow. The same phenomenon is observed on Windows, going from 3.43s to 7.03s. I was not expecting this result at all. My hypothesis is that the repeated integer-to-float conversions, combined with the fact that the compiler may not have successfully vectorized the code using FMAD, introduced more overhead than actual gain.

CPU usage remains close to 100%, confirming that the computation is still entirely CPU-bound. Memory usage did not change significantly compared to optimization 1, which makes sense since float occupies the same space as int on my system.

To conclude, this experience taught me that a low-level hardware optimization does not always deliver the expected results. The effectiveness of AVX units heavily depends on how well the compiler exploits these instructions, and in a single-threaded context, this potential seems difficult to fully unlock. This makes me curious to see whether moving to multithreading will make a real difference.

Task 1.3: Compute a matrix sequentially with the three optimizations

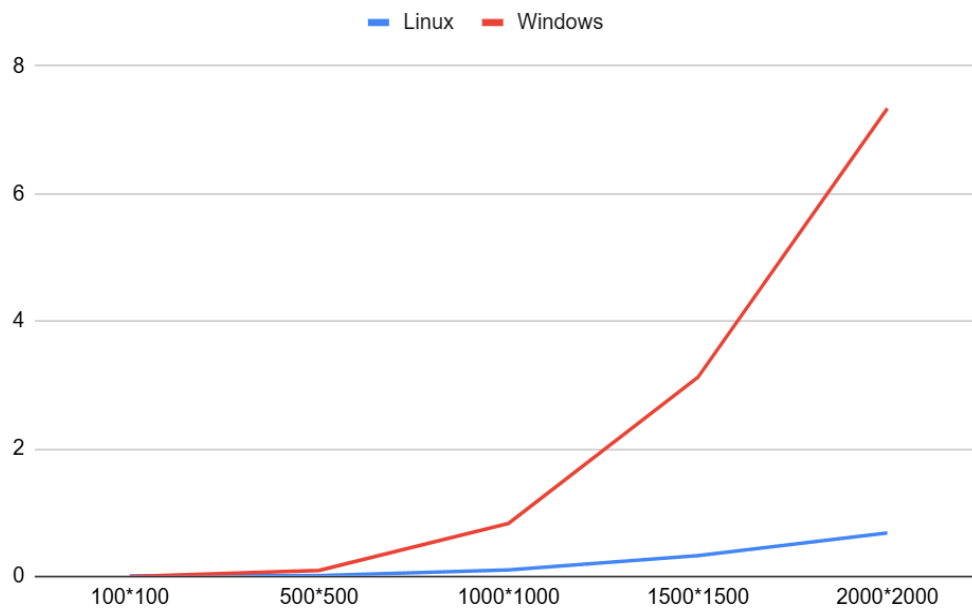
Value table under Debian Linux

Tour Ordi Debian Linux			
Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.00851074	188.07	5.6
500*500	0.0144261	897.35	10
1000*1000	0.103683	903.03	24
1500*1500	0.327532	955.26	42
2000*2000	0.682965	1126.55	65

Value table under Windows

Tour Ordi Windows			
Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.0004578	0	5.1
500*500	0.095411	97.11	8.7
1000*1000	0.832444	78.83	20
1500*1500	3.12302	88.06	39
2000*2000	7.33667	82.21	66

Graphical Representation of Data



Commentary

In this task, I applied all three optimizations together to the sequential matrix computation: the transposed matrix BT, the conversion to floats for FMAD/AVX, and finally the 64 byte memory alignment using “_mm_malloc”. This last optimization ensures that the data is correctly aligned in memory, which should theoretically allow the CPU to access it more efficiently.

The results here are honestly remarkable and this time, in a very positive way. On Linux, for a 2000×2000 matrix, I went from 6.85s (optimization 2) all the way down to 0.68s, which is a speedup of roughly 10x compared to the previous task, and even about 5x faster than my best result so far (3.14s with optimization 1 alone). This is by far the best sequential result I have obtained throughout this lab.

What surprised me the most is how dramatically Linux outperforms Windows in this configuration. On Windows, the same 2000×2000 matrix takes 7.33s, more than 10 times slower than Linux. I did not expect such a massive gap. My interpretation is that the memory alignment optimization interacts much more effectively with the Linux kernel's memory management and the GCC compiler's ability to fully vectorize the code with AVX instructions. On Windows, it seems like the alignment either has less impact or the compiler does not exploit it as efficiently.

CPU usage on Linux is also very interesting, it goes well above 100%, reaching up to 1126% for a 2000×2000 matrix. This tells me that the compiler successfully parallelized part of the computation using AVX vector units, effectively processing multiple floating-point operations simultaneously within a single thread. On Windows, CPU usage stays below 100%, which confirms that the vectorization is not being triggered in the same way.

Memory usage remains consistent with the previous optimizations, scaling as expected with N^2 . To conclude, this experience showed me that memory alignment is not just a minor detail, it can be the key factor that unlocks the full potential of AVX vectorization. The combination of all three optimizations on Linux delivered results I genuinely did not anticipate, and it made me realize how much the OS and compiler environment matter when pushing hardware to its limits.

Task 2: Compute a matrix case by case with thread

Value table under Debian Linux

TP2 Version - Code (linux)			
Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.23	127.74	86
500*500	/	/	/
1000*1000	/	/	/
1500*1500	/	/	/
2000*2000	/	/	/

Value table under Windows

TP2 Version - Code (Windows)			
100*100	0,58	40,23	110
500*500	568.00	153,28	1600
1000*1000	749.00	114,21	2100
1500*1500	5126.65	54.9	1200
2000*2000	/	/	/

Commentary

In this exercise, I attempted to parallelize the matrix computation by creating one thread per element (cell) of matrix C, which implies launching N^2 threads simultaneously. This approach quickly allowed me to observe its impracticality as I increase the matrix size.

For a 100×100 matrix, my Linux version already takes 0.23s with a CPU usage of 127.74% and a memory consumption of 86 Mo — which is already significantly worse than my sequential version that only needed 0.0044s for the same computation. This observation confirms to me that creating 10,000 threads simultaneously for a 100×100 matrix generates considerable overhead in terms of thread creation and management, which completely cancels out any potential benefit from parallelism.

The situation becomes even more alarming on Windows, where my 500×500 matrix (250,000 threads) takes 568s and consumes up to 1600 Mo of memory, and my 1000×1000 matrix (1,000,000 threads) reaches 749s with 2100 Mo. Beyond these sizes, my program simply crashed in both environments, which I attribute to system limitations on the maximum number of threads that can be created simultaneously and certainly regarding the memory that can be allocated.

These results directly motivate me to adopt a more reasonable approach in the next task, where I will use one thread per row instead of one thread per element. This decision will allow me to drastically reduce the number of threads from N^2 to N , while maintaining a significant and balanced workload per thread.

Task 3: Compute a matrix row by row with thread

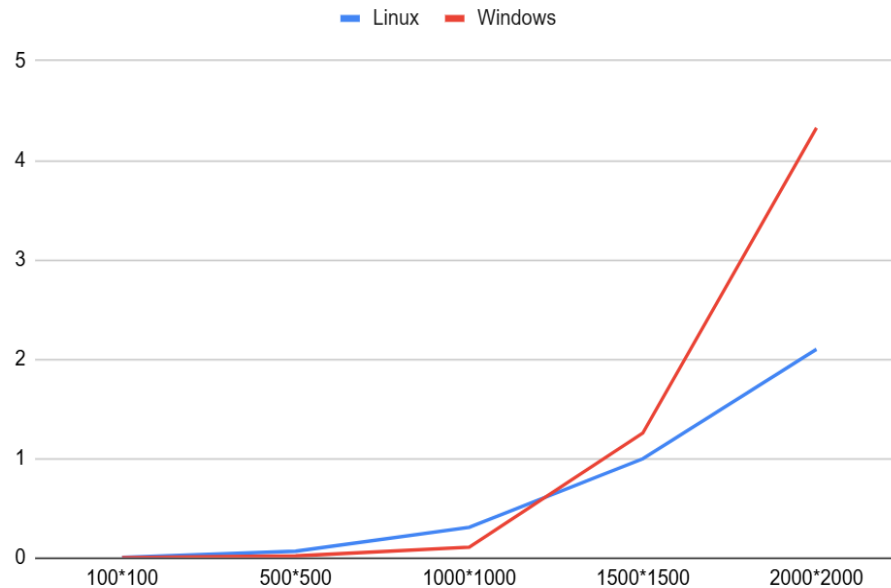
Value table under Debian Linux

TP3 Matrix : Version - Code (Linux)			
Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.0087	246.3	5.7
500*500	0.07	641.59	13
1000*1000	0.31	1206.27	24
1500*1500	1	1278.22	44
2000*2000	2.1	147.94	70

Value table under Windows

TP3 Matrix : Version - Code (Windows)			
Matrix size	Time [s]	CPU usage [%]	Memory usage [Mo]
100*100	0.0051325	0	7.1
500*500	0.0224664	69.55	14
1000*1000	0.111073	379.82	29
1500*1500	1.25921	1342.61	46
2000*2000	4.32547	1440.95	67

Graphical Representation of Data



Commentary

In this exercise, I parallelized the matrix computation by assigning one thread per row of matrix C, which means launching exactly N threads simultaneously instead of the N^2 threads from the previous task. This decision proved to be far more reasonable and the results clearly reflect it.

Comparing my two environments, I observe that Linux and Windows deliver much more similar performance than in previous tasks. For a 2000×2000 matrix, I obtain 2.1s on Linux versus 4.32s on Windows, a gap that remains consistent across all matrix sizes. Unlike Task 2 where my program crashed beyond 1000×1000 , I was here able to complete all my measurements up to 2000×2000 without any instability, which already demonstrates the superiority of this approach in terms of robustness.

Compared to my sequential version (Task 1: 16.02s for 2000×2000 on Linux), this approach reduces execution time to 2.1s, a speedup of approximately 8x. Compared to my per-element threading approach (Task 2), the improvement is even more dramatic since that version crashed entirely beyond 1000×1000 .

CPU usage increases significantly, reaching over 1200% in both environments for large matrices, which confirms that multiple cores are now being effectively utilized in parallel. Memory usage remains stable and modest, scaling predictably with matrix size without the explosive growth I observed in Task 2.

These results convince me that one thread per row represents a much better trade-off between parallelism and overhead. However, I notice that the number of threads remains fixed at N, which means that for a 2000×2000 matrix I am launching 2000 threads, far more than the 16 logical threads available on my CPU...

Task 4: Dynamic number of the threads

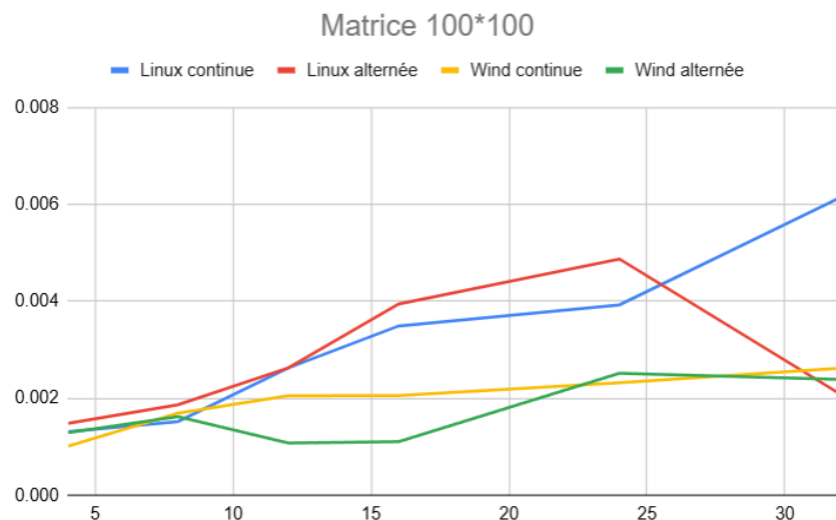
Value table under Debian Linux

Ordi tour linux alternée					Ordi tour linux continue				
Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]	Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]
100×100	4	0.00148126	304.13	2.8	100×100	4	0.001307	308.12	2.8
	8	0.001868	298.37	3.6		8	0.00151954	342.47	3.5
	12	0.002627	672.61	3.6		12	0.002627	266.97	5.5
	16	0.003945	450.63	5.7		16	0.003492	236.33	5.5
	24	0.004874	219.08	5.7		24	0.003928	187.69	5.6
	32	0.002086	227.17	5.7		32	0.006141	187.62	5.6
500×500	4	0.021986	390.91	6.2	500×500	4	0.01607	390.75	6.2
	8	0.01593	681.3	6.2		8	0.01568	630.72	6.2
	12	0.0180552	924.87	6.2		12	0.01691	909	6.2
	16	0.0179083	1368.96	6.4		16	0.0164853	1087.18	6.3
	24	0.0187342	1024.4	6.4		24	0.01759	1016.01	6.3
	32	0.0213678	1194.22	6.4		32	0.02369	906.43	8.1
1000×1000	4	0.152343	395.55	14	1000×1000	4	0.1788	397.68	14
	8	0.0926059	778.28	15		8	0.08171	765.36	15
	12	0.0974964	922.88	15		12	0.09226	936.1	15
	16	0.0745679	1509.19	15		16	0.07618	1413.99	15
	24	0.0839683	1281.27	15		24	0.08359	1230.05	15
	32	0.076551	1371.96	15		32	0.08897	1172.69	15
1500×1500	4	0.462156	397.89	29	1500×1500	4	0.46205	398.31	29
	8	0.255965	763.73	29		8	0.244025	781.13	29
	12	0.276835	958.74	29		12	0.280854	958.74	29
	16	0.220076	1509.01	29		16	0.226256	1461.18	29
	24	0.249732	1351.15	29		24	0.24047	1307.81	29
	32	0.254831	1360.68	29		32	0.22468	1451.23	29
2000×2000	4	1.31993	398.6	49	2000×2000	4	1.26802	396.68	49
	8	0.603367	795.85	49		8	0.63707	773.98	49
	12	0.634152	960.37	49		12	0.639637	1012.46	49
	16	0.518796	1467.05	49		16	0.551521	1453.5	49
	24	0.612459	1344.53	49		24	0.572752	1301.04	49
	32	0.57016	1,407.22	49		32	0.569306	1392.13	49

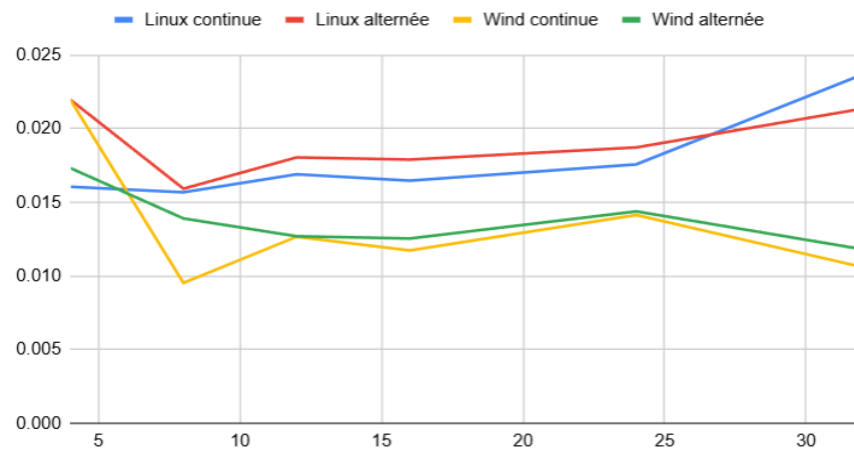
Value table under Windows

Ordi Tour Windows alternée					Ordi Tour Windows continue				
Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]	Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]
100×100	4	0.001289	0	4.7	100×100	4	0.0010091	0	4.6
	8	0.0016207	0	4.8		8	0.0016921	0	4.7
	12	0.0010762	0	4.8		12	0.0020527	0	4.8
	16	0.0011044	0	4.9		16	0.002056	0	4.8
	24	0.0025158	0	5		24	0.0023201	0	4.9
	32	0.0023887	0	5.1		32	0.0026209	0	5.1
500×500	4	0.017334	0	7.4	500×500	4	0.0219978	71.03	7.4
	8	0.0139094	112.33	7.5		8	0.0095327	0	7.5
	12	0.0126989	861.3	7.5		12	0.0126702	246.64	7.5
	16	0.0125468	0	7.6		16	0.011743	137.17	7.6
	24	0.0143898	0	7.6		24	0.0141355	110.54	7.6
	32	0.0118134	132.27	7.7		32	0.0105959	147.46	7.7
1000×1000	4	0.201267	279.48	16	1000×1000	4	0.198189	291.7	16
	8	0.125845	496.64	16		8	0.115654	418.81	16
	12	0.104167	600	16		12	0.127542	869.81	16
	16	0.115451	1177.45	16		16	0.112002	1171.86	16
	24	0.113091	1008.59	16		24	0.120543	920.31	16
	32	0.0999689	1140.98	16		32	0.0975313	1185.52	16
1500×1500	4	0.781483	289.91	30	1500×1500	4	0.756815	278.72	30
	8	0.521154	515.68	30		8	0.530079	557.11	30
	12	0.837978	753.3	30		12	0.610311	762.93	30
	16	1.24137	1124.01	30		16	1.35822	1354.02	30
	24	1.36926	1396.74	30		24	1.39937	1356.64	30
	32	1.32993	1358.16	30		32	1.40992	1355.36	30
2000×2000	4	4.66138	268.5	50	2000×2000	4	4.49626	296.08	50
	8	3.50743	441.92	50		8	3.4226	505.83	50
	12	3.87904	774.6	50		12	3.72205	753.95	50
	16	4.34176	1329.03	50		16	4.27581	1199.33	50
	24	4.23679	1439.77	50		24	4.23444	1409.2	50
	32	4.27724	1483.87	50		32	4.40466	1354.04	50

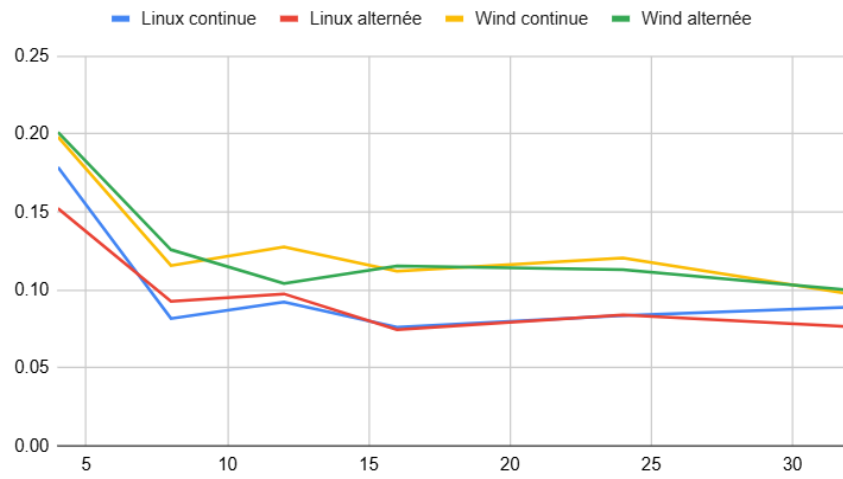
Graphical Representation of Data



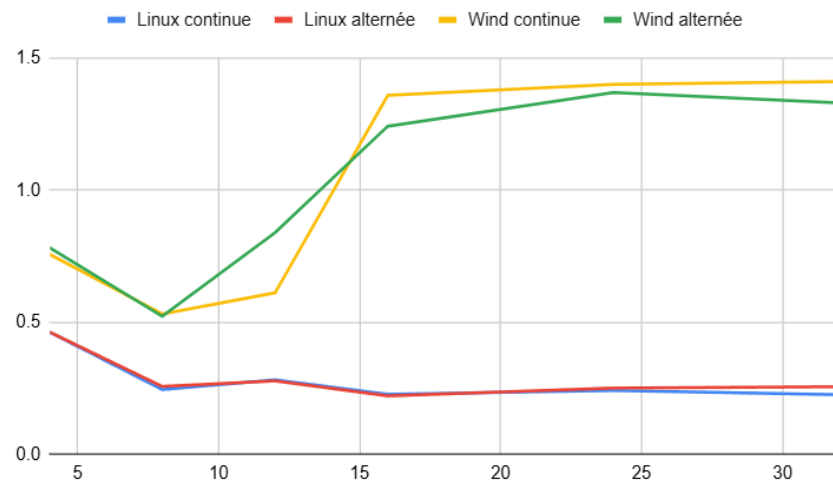
Matrice 500*500



Matrice 1000*1000

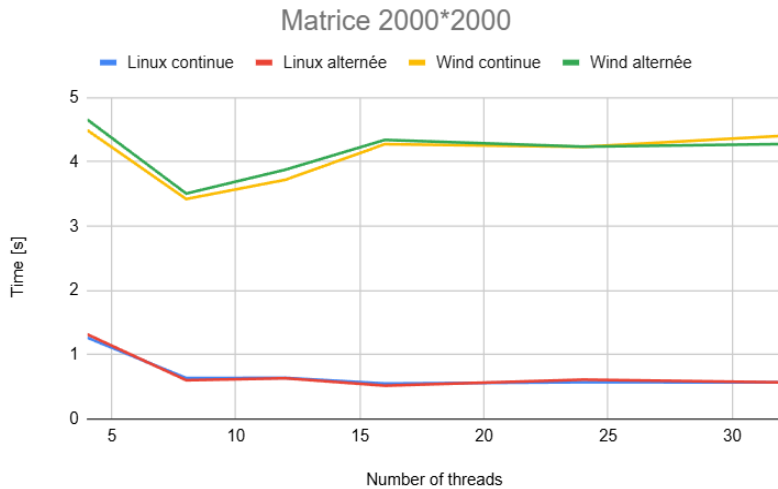


Matrice 1500*1500



Commentary

In this task, I parallelized the matrix computation by letting the user choose the number of threads to use, which allowed me to explore the direct impact of thread count on performance and find the optimal configuration for my hardware.



For both distributions (alternated and contiguous), execution time decreases significantly between 4 and 16 threads, my CPU having 16 logical threads, then stabilizes or even increases slightly beyond that point. This phenomenon is explained by context switching: beyond $T=16$ logical threads, my CPU can no longer execute all threads simultaneously and must alternate between them rapidly, generating an overhead that cancels out the benefit of additional parallelism.

For a 2000×2000 matrix on Linux, I go from 4.66s with 4 threads to 0.518s with 16 threads, a speedup of approximately 9x, before slightly increasing back to 0.570s with 32 threads, confirming that $T=16$ is indeed the optimal number of threads for my machine.

Comparing the two distribution modes, I observe that performance is very similar between the alternated and contiguous versions across all matrix sizes and thread counts. This similarity is explained by the fact that both approaches distribute an equivalent workload per thread, the difference lies only in the order in which rows are assigned, which has no major impact on cache efficiency in this context.

Linux confirms its superiority over Windows for all tested configurations. For a 2000×2000 matrix with 16 threads, I obtain 0.518s on Linux versus 4.34s on Windows, a factor of 8x, which I attribute to differences in thread scheduling and memory management between the two operating systems, as observed in previous tasks.

CPU usage increases proportionally with the number of threads up to $T=16$, reaching over 1300% on Linux for large matrices, confirming that all 16 of my logical cores are effectively being utilized. Memory usage remains stable and predictable at N^2 , regardless of the number of threads used.

These results allow me to conclude that the optimal number of threads for my CPU is equal to the number of available logical threads ($T=16$) and that exceeding this value introduces counterproductive context switching overhead, exactly as the course explains with the values $[0.25T, 0.5T, 0.75T, T, 1.5T, 2T]$.

Task 4.1: second optimization

In this task, I had to upgrade my code by introducing the transposition of matrix B. Instead of reading the columns of B which causes memory jumps and therefore numerous cache misses. I now read the rows of BT sequentially. Since modern CPUs load data in cache lines, accessing memory sequentially dramatically reduces the number of cache misses and improves data locality. This simple change in memory access pattern is expected to deliver a speedup of approximately 2 to 3 times compared to the naive version.

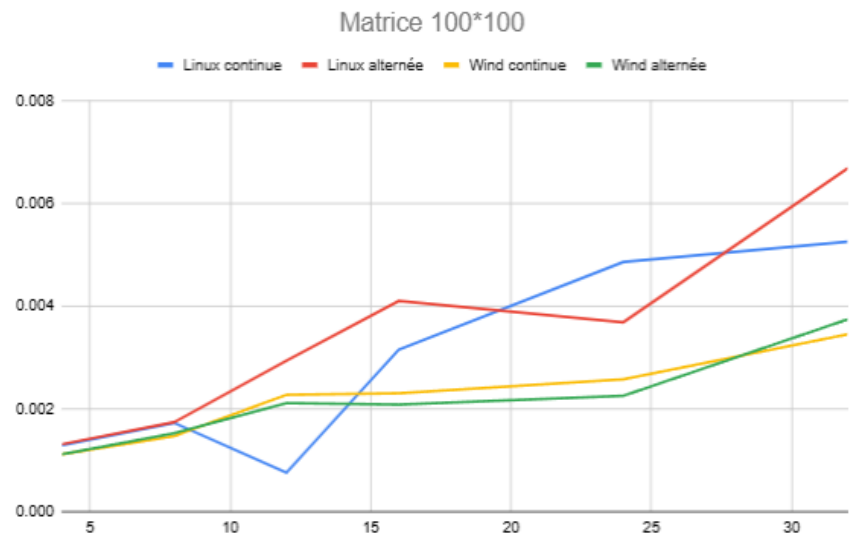
Value table under Debian Linux

Ordi tour linux alternée					Ordi tour linux continue				
Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]	Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]
100×100	4	0.00131113	292.73	3.3	100×100	4	0.00129193	303.89	3.3
	8	0.00173992	292.89	3.5		8	0.00172705	314.99	3.5
	12	0.002934	231.2	5.3		12	0.000756002	228.97	5.5
	16	0.00410075	200.16	5.6		16	0.0031546	225.13	5.3
	24	0.0036852	238.82	5.4		24	0.00485873	201.74	5.4
	32	0.00688593	188.2	5.4		32	0.00525429	199.67	5.3
500×500	4	0.0174889	395.18	6.9	500×500	4	0.01918	395.67	6.9
	8	0.0101071	715.91	7.2		8	0.0254765	687.63	7.2
	12	0.0011566	1042.47	7.1		12	0.0209637	1045.76	7.2
	16	0.0128146	1103.4	7.3		16	0.0121674	1182.44	7.2
	24	0.0146117	795.14	7.3		24	0.0248713	1202.89	7.3
	32	0.0125676	1052.27	7.3		32	0.0126688	1099.29	7.4
1000×1000	4	0.099825	393.68	18	1000×1000	4	0.108808	397.83	18
	8	0.053684	789.66	19		8	0.0560795	767.37	19
	12	0.0599079	969.85	19		12	0.0588028	992.94	19
	16	0.0493582	1417.96	19		16	0.0592167	1449.22	19
	24	0.0557561	1219.33	19		24	0.0694185	1264.08	19
	32	0.051159	1316.5	19		32	0.0504118	1376.22	19
1500×1500	4	0.319458	397.89	38	1500×1500	4	0.310199	399.56	37
	8	0.163774	787.06	38		8	0.164207	786.3	38
	12	0.1733	989.1	38		12	0.177687	988.64	38
	16	0.161152	1517.87	38		16	0.154751	1464.36	38
	24	0.16744	1306.92	38		24	0.160498	1334.93	38
	32	0.173619	1369.74	38		32	0.164708	1322.17	38
2000×2000	4	0.797087	398.6	64	2000×2000	4	0.789012	399.46	64
	8	0.59045	648.25	64		8	0.404319	794.3	64
	12	0.400032	1042.2	64		12	0.424007	986.94	64
	16	0.349939	1531.12	64		16	0.349995	1532.76	64
	24	0.383654	1326.72	64		24	0.389234	1280.58	65
	32	0.381149	1399.02	65		32	0.378468	1404.85	65

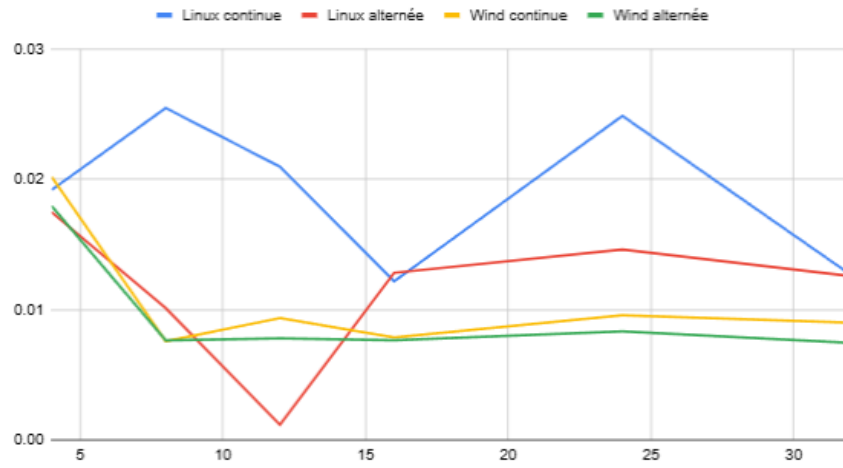
Value table under Windows

Ordi Tour Windows alternée					Ordi Tour Windows continue				
Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]	Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]
100×100	4	0.001114	0	5.1	100×100	4	0.0011145	0	4.7
	8	0.0015249	0	4.8		8	0.0014686	0	4.8
	12	0.0021116	0	4.9		12	0.0022736	0	4.8
	16	0.002083	0	4.9		16	0.0023026	0	4.9
	24	0.0022533	0	4.9		24	0.0025747	0	5
	32	0.0037436	0	5.1		32	0.0034515	0	5.2
500×500	4	0.0179563	87.02	8.4	500×500	4	0.0201978	154.72	8.4
	8	0.0076337	1228.11	8.5		8	0.0075657	0	8.5
	12	0.0077881	1404.39	8.5		12	0.0093465	0	8.5
	16	0.0076433	0	8.6		16	0.0078593	198.81	8.6
	24	0.008326	375.33	8.8		24	0.0095618	0	8.8
	32	0.0074416	0	8.9		32	0.0089921	0	8.9
1000×1000	4	0.106491	234.76	20	1000×1000	4	0.103407	302.2	20
	8	0.0678639	437.46	20		8	0.0649893	456.81	20
	12	0.0519517	541.37	20		12	0.0519914	631.11	20
	16	0.048507	1062.99	20		16	0.0487144	962.24	20
	24	0.056986	630.64	20		24	0.0604433	672.12	20
	32	0.053104	853.25	20		32	0.050646	1048.95	20
1500×1500	4	0.36596	294.6	39	1500×1500	4	0.39645	279.83	39
	8	0.210963	525.86	39		8	0.210611	511.9	39
	12	0.168371	807.37	39		12	0.169677	764.32	39
	16	0.149234	1298.29	39		16	0.15171	1390.4	39
	24	0.167207	1055.95	39		24	0.165758	1112.31	39
	32	0.159008	1287.28	39		32	0.168391	1187.71	39
2000×2000	4	0.932848	329.97	66	2000×2000	4	0.987702	300.57	66
	8	0.626097	369.35	66		8	0.534282	505.95	66
	12	0.429875	806.92	66		12	0.414806	847.54	66
	16	0.382589	1372.23	66		16	0.360335	1417.95	66
	24	0.399507	1235.9	66		24	0.387319	1290.92	66
	32	0.378238	1342.58	66		32	0.405065	1168.79	66

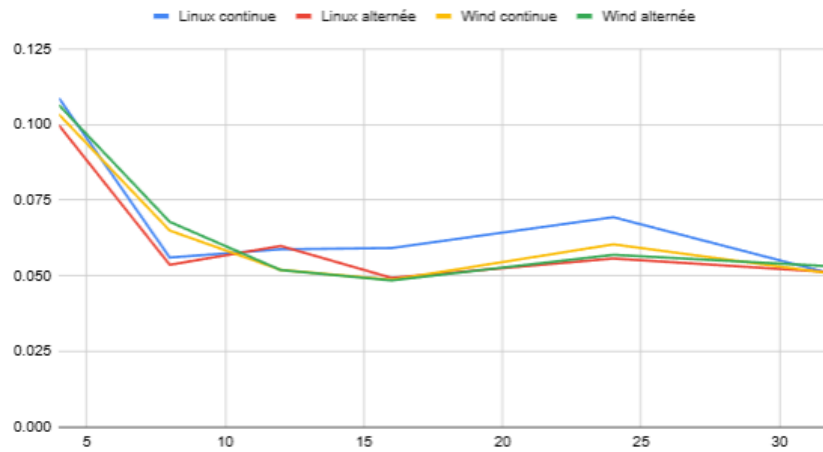
Graphical Representation of Data



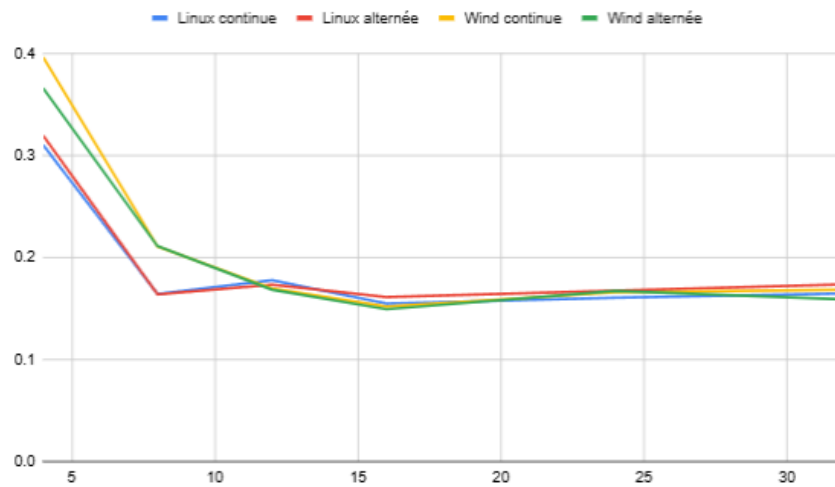
Matrice 500*500

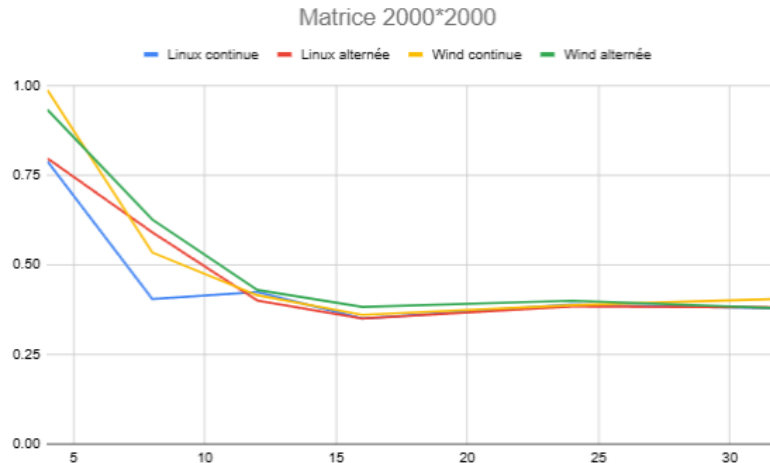


Matrice 1000*1000



Matrice 1500*1500





Commentary

In this task, I applied the first optimization to my multithreaded matrix computation by introducing the transposed matrix BT, replacing the column-access pattern of B with a row-access pattern on BT. This eliminates the cache misses caused by jumping N positions in memory at each iteration of the inner loop, replacing them with fully sequential memory reads on both A and BT simultaneously.

Comparing these results with my unoptimized TP4, the improvement is immediately visible. For a 2000×2000 matrix on Linux with 16 threads, I went from 0.518s (unoptimized) to 0.349s (optimized), representing a speedup of approximately 1.5x brought solely by the memory access optimization, without any change to the threading strategy. This confirms what I had already observed in the sequential tasks: cache-friendly memory access has a measurable and consistent impact regardless of the level of parallelism applied.

The optimal number of threads remains $T=16$ for all matrix sizes and both distribution modes, consistent with my previous observations. Beyond 16 threads, execution time stabilizes or slightly increases due to context switching overhead, exactly as predicted by the course with the values $[0.25T, 0.5T, 0.75T, T, 1.5T, 2T]$.

Comparing the two distribution modes, alternating and contiguous performance remain very close across all configurations, confirming that the row assignment order has no significant impact on cache efficiency when BT already ensures sequential memory access.

Linux continues to outperform Windows significantly. For a 2000×2000 matrix with 16 threads, I obtain 0.349s on Linux versus 0.382s on Windows, a much smaller gap than in previous tasks, which suggests that the cache optimization partially compensates for the OS-level overhead differences between the two environments.

Task 4.2: second optimization

In this task, I had to upgrade my code by converting the data type from integers to floats. Even though both float and int are 32 bit types and occupy the same amount of memory, the key difference lies in how the CPU handles them.

The AVX units of my processor are specifically designed to work with floating-point values, and more importantly, they support the FMAD instruction, which performs both the multiplication and the addition in a single instruction instead of two separate ones. This reduces the total number of instructions executed and allows the CPU to process more operations per cycle. Combined with the memory access improvement from the first optimization, the use of FMAD through AVX units is expected to deliver an additional speedup of approximately 1.5x.

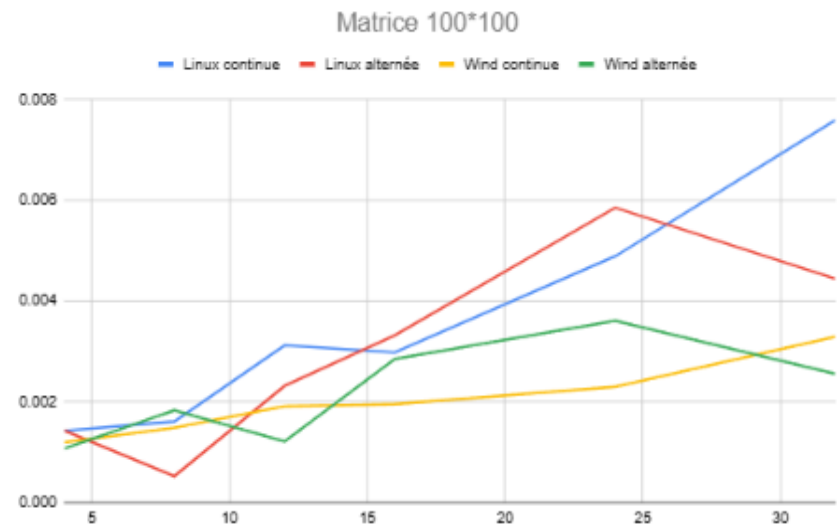
Value table under Debian Linux

Ordi tour linux alternée					Ordi tour linux continue				
Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]	Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]
100×100	4	0.00142679	328.36	3.3	100×100	4	0.00142055	329.59	3.3
	8	0.000521995	402.49	3.5		8	0.00160637	392.06	3.5
	12	0.0023137	332.97	3.5		12	0.00311856	267.17	5.3
	16	0.00331865	268.42	5.3		16	0.00297852	270.11	5.3
	24	0.00584894	201.92	5.4		24	0.00488301	232.95	5.4
	32	0.00443538	244.38	5.3		32	0.00758771	183.19	5.7
500×500	4	0.0335652	390.06	6.9	500×500	4	0.0338955	388.34	6.9
	8	0.017764	764.73	7.1		8	0.0185399	732.52	7.1
	12	0.0141053	1136.39	7.2		12	0.015225	1033.88	7.2
	16	0.0145842	1115.13	7.2		16	0.0166931	1019.11	7.2
	24	0.017715	974.82	7.3		24	0.0179203	1031.71	7.3
	32	0.0147781	1157.32	7.3		32	0.0179863	927.72	7.4
1000×1000	4	0.207169	399.53	18	1000×1000	4	0.208048	395.6	18
	8	0.109945	785.15	19		8	0.114301	763.61	19
	12	0.0813654	1109.89	19		12	0.0800236	1132.91	19
	16	0.0722935	1281.55	19		16	0.0721307	1274.96	19
	24	0.0754996	1190.21	19		24	0.0745813	1227.15	19
	32	0.072313	1271.81	19		32	0.0676832	1375.66	19
1500×1500	4	0.717309	396.87	37	1500×1500	4	0.702343	399.83	37
	8	0.363827	792.45	38		8	0.366895	784.92	38
	12	0.24846	1189.44	38		12	0.247881	1188.5	38
	16	0.203776	1459.74	38		16	0.209301	1427.85	38
	24	0.221567	1345.39	38		24	0.216737	1377.07	38
	32	0.218154	1369.16	38		32	0.239677	1245.79	38
2000×2000	4	1.77634	399.91	64	2000×2000	4	1.80499	399.05	64
	8	0.904359	789.86	64		8	0.924552	782.65	64
	12	0.594541	1188.85	64		12	0.624762	1148.77	64
	16	0.461672	1555.86	64		16	0.577094	1258.94	64
	24	0.52017	1372.69	64		24	0.550992	1334.82	64
	32	0.503332	1406.87	69		32	0.512299	1398.29	65

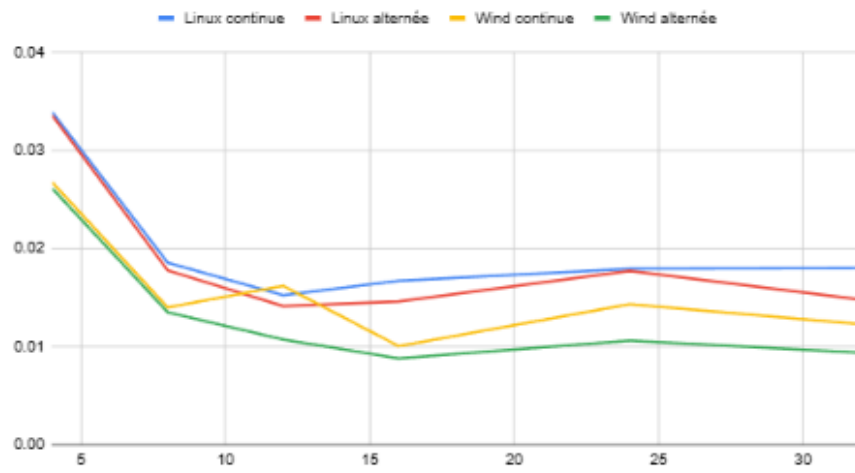
Value table under Windows

Ordi Tour Windows alternée					Ordi Tour Windows continue				
Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]	Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]
100×100	4	0.0010672	0	5.1	100×100	4	0.0011915	0	4.7
	8	0.001827	0	4.8		8	0.0014807	0	4.8
	12	0.00121	0	4.9		12	0.0019086	0	4.8
	16	0.0028464	0	4.9		16	0.0019501	0	4.9
	24	0.0036043	0	5		24	0.0022937	0	5
	32	0.0025497	0	5.2		32	0.0032892	0	5.1
500×500	4	0.0261151	239.33	8.4	500×500	4	0.0267336	233.79	8.4
	8	0.0134838	0	8.5		8	0.0139785	0	8.4
	12	0.0107057	0	8.5		12	0.0161771	96.59	8.5
	16	0.0087838	177.88	8.6		16	0.0100305	1557.75	8.6
	24	0.0105943	147.48	8.8		24	0.0143081	1201.25	8.8
	32	0.0093666	0	8.9		32	0.0122732	0	8.9
1000×1000	4	0.221153	289.68	20	1000×1000	4	0.212009	324.28	20
	8	0.108214	519.8	20		8	0.110439	481.03	20
	12	0.0739522	760.63	20		12	0.0772744	525.72	20
	16	0.0655521	1072.62	20		16	0.067242	999.19	20
	24	0.0745674	859.12	20		24	0.0907434	723.19	20
	32	0.0825579	813.82	20		32	0.0691654	948.81	20
1500×1500	4	0.740797	320.6	39	1500×1500	4	0.732335	309.37	39
	8	0.366025	601.9	39		8	0.364583	621.43	39
	12	0.274408	837.03	39		12	0.251819	688.74	39
	16	0.206726	1234.9	39		16	0.215312	1378.81	39
	24	0.254441	1025.53	39		24	0.26755	1010.32	39
	32	0.218237	1281.28	39		32	0.255686	1014.43	40
2000×2000	4	1.84847	300.92	66	2000×2000	4	1.83473	288.7	66
	8	0.896697	590.71	66		8	0.958603	508.55	66
	12	0.618053	755.9	66		12	0.640667	760.93	66
	16	0.529756	1315.43	66		16	0.493581	1421.37	66
	24	0.574608	1169.28	66		24	0.539682	1294.17	66
	32	0.51101	1327.03	66		32	0.564811	1156.36	66

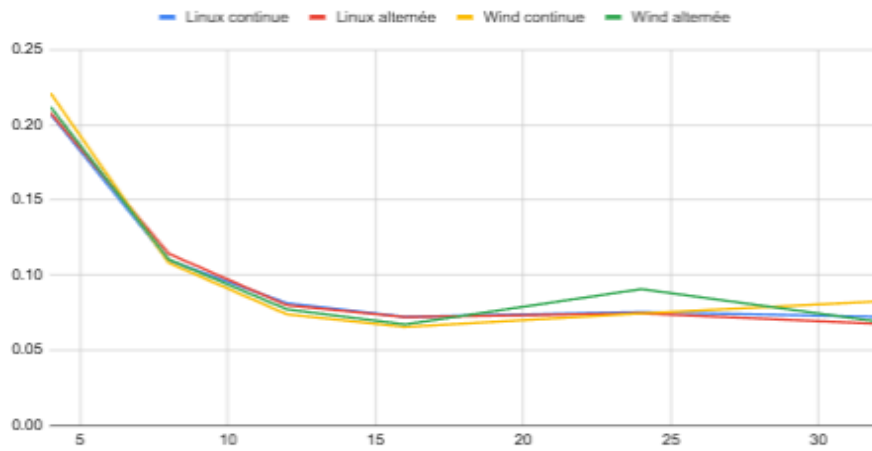
Graphical Representation of Data



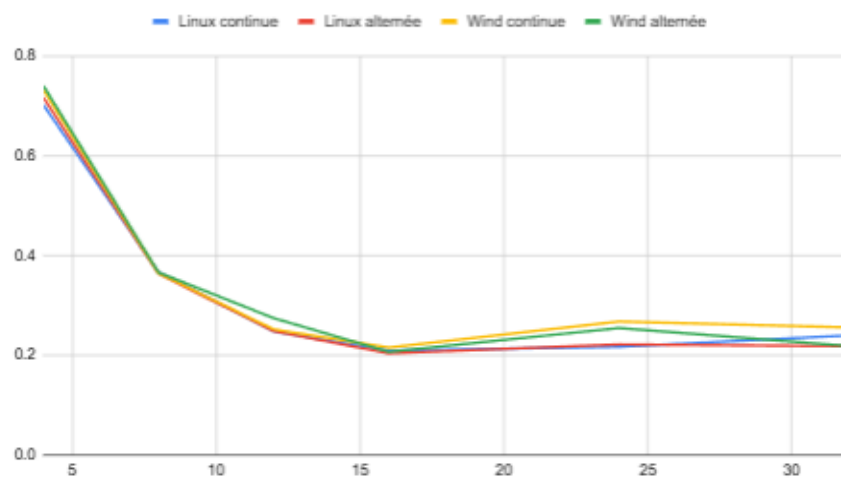
Matrice 500*500



Matrice 1000*1000

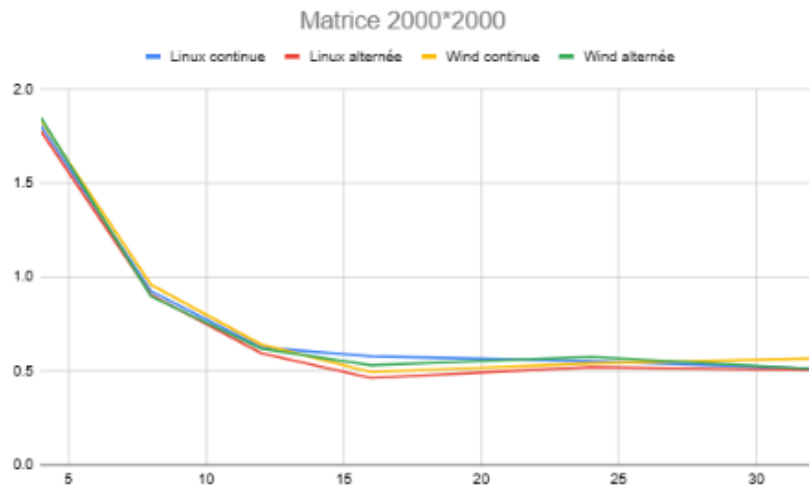


Matrice 1500*1500



Commentary

In this task, I applied the second optimization to my multithreaded matrix computation by converting the data from int to float, allowing the CPU to exploit the AVX units and the FMAD instruction, which performs both the multiplication and the addition in a single instruction instead of two.



Comparing these results with my TP4 optimization 1, the improvement is not as straightforward as I expected. For a 2000×2000 matrix on Linux with 16 threads, I went from 0.349s (optim 1) to 0.461s. I noticed that some values are actually slightly higher than with optimization 1, which I think can be explained by the fact that the compiler did not always automatically vectorize the code in an optimal way without the appropriate compilation flags. This is something I would want to investigate further by explicitly enabling AVX flags such as `-mavx2` or `-mfma` during compilation.

The optimal number of threads remains consistent at $T=16$ for all matrix sizes and both distribution modes. Beyond 16 threads, execution time stabilizes or slightly increases due to context switching overhead, exactly as predicted by the course with the values $[0.25T, 0.5T, 0.75T, T, 1.5T, 2T]$.

Comparing the two distribution modes, alternating and contiguous performance remain very close across all configurations, once again confirming that the row assignment order has no significant impact on overall performance.

Linux continues to outperform Windows across all tested configurations. For a 2000×2000 matrix with 16 threads, I obtain 0.461s on Linux versus 0.529s on Windows, a relatively small gap, suggesting that the AVX/FMAD optimization benefits both environments in a comparable way.

CPU usage remains high and proportional to the number of threads up to $T=16$, reaching over 1500% on Linux for large matrices. Memory usage remains stable and predictable at N^2 , identical to the previous optimization since float and int both occupy 4 bytes in memory.

These results confirm that the combination of all three elements sequential memory access via BT, exploitation of AVX units via float, and an optimal thread count of $T=16$ represents the most effective strategy for parallel matrix multiplication on my hardware.

Task 4.3: optimisation

The CPU loads memory in blocks of 64 bytes (a cache line). If the data is not aligned on 64 bytes, a single element can end up split across two cache lines, forcing the CPU to perform two memory loads instead of one. By correctly aligning the data with “_mm_malloc”, every cache line load is fully utilized, avoiding any wasted reads. This alignment optimization delivers an additional speedup of approximately 1.2x.

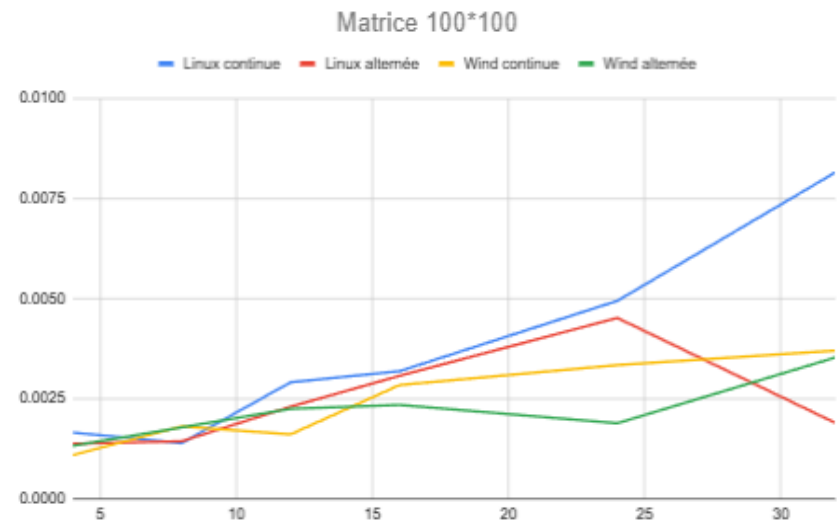
Value table under Debian Linux

Ordi tour linux alternée					Ordi tour linux continue				
Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]	Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]
100×100	4	0.00137478	326.16	3.3	100×100	4	0.00165351	299.24	3.3
	8	0.00143611	434.86	3.5		8	0.00139106	429.17	3.5
	12	0.0022996	356.89	3.5		12	0.00290933	257.72	5.5
	16	0.00307049	286.27	5.3		16	0.0031865	263.93	5.3
	24	0.00451729	240.85	5.4		24	0.00494439	233.98	5.4
	32	0.0018952	211.76	5.4		32	0.00615157	183.77	5.5
500×500	4	0.0302082	396.57	6.9	500×500	4	0.0434062	390.13	6.9
	8	0.0171686	763.71	7.2		8	0.0288755	764.97	7.2
	12	0.0136837	1133.67	7.2		12	0.0237935	1080.05	7.2
	16	0.024717	1156.64	7.2		16	0.0275954	1097.63	7.3
	24	0.0254149	1106.49	7.3		24	0.0237497	1122.27	7.3
	32	0.0262846	1339.96	7.4		32	0.0132382	996.24	7.3
1000×1000	4	0.230891	389.22	18	1000×1000	4	0.221673	399.22	18
	8	0.123332	769.83	19		8	0.089203	1145.57	19
	12	0.0787277	1165.43	19		12	0.0873452	1138.91	19
	16	0.0849579	1218.7	19		16	0.0836089	1275.11	19
	24	0.076218	1310.16	19		24	0.0874488	1217.41	19
	32	0.0643301	1353.74	19		32	0.0788839	1329.5	19
1500×1500	4	0.765406	388.92	37	1500×1500	4	0.736138	399.72	37
	8	0.368685	789.29	38		8	0.394974	775.96	38
	12	0.248049	1190.49	38		12	0.265184	1166.9	38
	16	0.204413	1486.28	38		16	0.253385	1232.68	38
	24	0.237926	1237.87	38		24	0.227255	1363.17	38
	32	0.220809	1357.92	38		32	0.265364	1176.62	38
2000×2000	4	1.82688	399.72	64	2000×2000	4	1.87526	387.85	64
	8	0.889734	793.17	64		8	0.92646	783.13	64
	12	0.625351	1168.13	64		12	0.63746	1148.97	64
	16	0.556958	1327.2	64		16	0.53033	1391.3	64
	24	0.569532	1301.81	68		24	0.571907	1284.76	65
	32	0.516492	1408.21	65		32	0.532579	1381.07	65

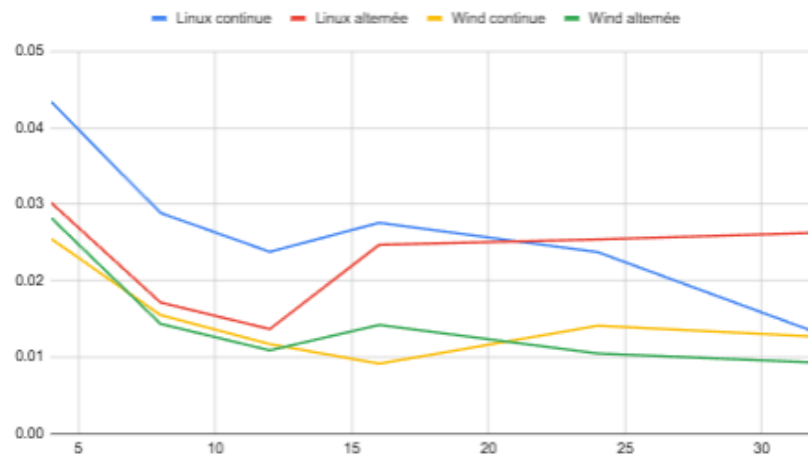
Value table under Windows

Ordi Tour Windows alternée					Ordi Tour Windows continue				
Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]	Matrix size	Number of threads	Time [s]	CPU usage [%]	Memory usage [Mo]
100×100	4	0.0013231	0	5.1	100×100	4	0.0010889	0	4.7
	8	0.001782	0	4.8		8	0.0018065	0	4.8
	12	0.0022469	0	4.8		12	0.0016079	0	4.9
	16	0.002344	0	5		16	0.0028402	0	4.9
	24	0.0018856	0	5		24	0.0033395	0	5
	32	0.0035278	0	5.1		32	0.0036999	0	5.2
500×500	4	0.0282189	221.48	8.4	500×500	4	0.0254967	183.58	8.4
	8	0.0143679	0	8.5		8	0.0155247	100.65	8.5
	12	0.0109055	0	8.6		12	0.0117197	133.62	8.5
	16	0.0142331	0	8.6		16	0.0091586	0	8.6
	24	0.0104939	1488.96	8.8		24	0.0141314	0	8.8
	32	0.0093047	0	8.9		32	0.0126974	123.06	8.9
1000×1000	4	0.213386	278.25	20	1000×1000	4	0.21358	321.89	20
	8	0.109577	570.37	20		8	0.109542	641.88	20
	12	0.0736862	805.78	20		12	0.0761197	677.39	20
	16	0.0752397	978.05	20		16	0.0629386	1216.48	20
	24	0.0746965	815.8	20		24	0.0637178	858.54	20
	32	0.068465	1163.92	20		32	0.0619355	839.07	20
1500×1500	4	0.746929	294.96	39	1500×1500	4	0.777858	285.24	39
	8	0.366904	587.69	39		8	0.370375	599.06	39
	12	0.245299	853.55	39		12	0.24471	919.45	39
	16	0.200631	1331.73	39		16	0.228681	1175.32	39
	24	0.25094	1008.71	39		24	0.255957	988.74	39
	32	0.218182	1310.54	39		32	0.254809	1079.52	39
2000×2000	4	1.8087	291.99	66	2000×2000	4	1.80581	275.14	66
	8	0.891989	611.34	66		8	0.910711	475.25	66
	12	0.6047	764.84	66		12	0.613945	778.77	66
	16	0.563031	1257.15	66		16	0.486943	1453.58	66
	24	0.56466	1202.59	66		24	0.537058	1277.21	66
	32	0.50889	1280.36	66		32	0.535531	1234.17	66

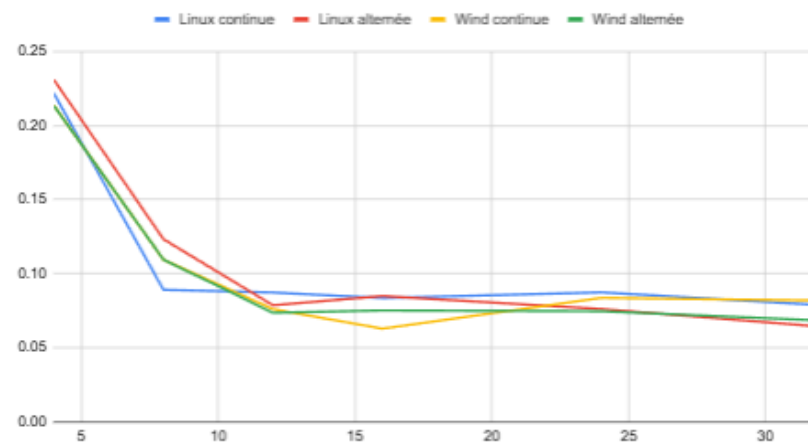
Graphical Representation of Data



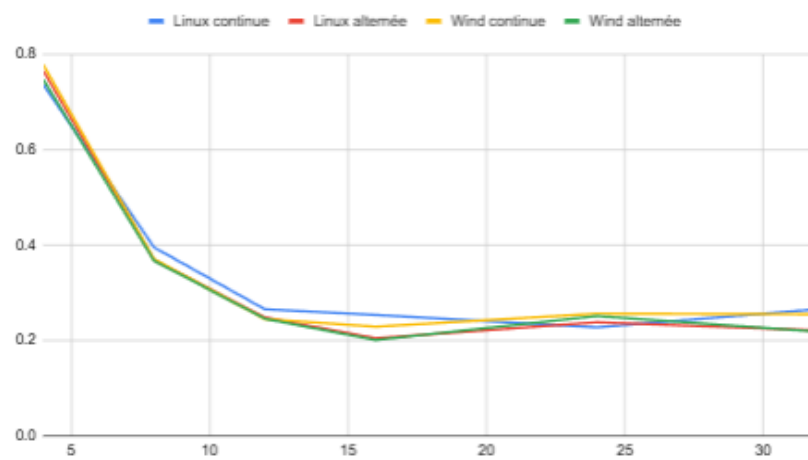
Matrice 500*500



Matrice 1000*1000

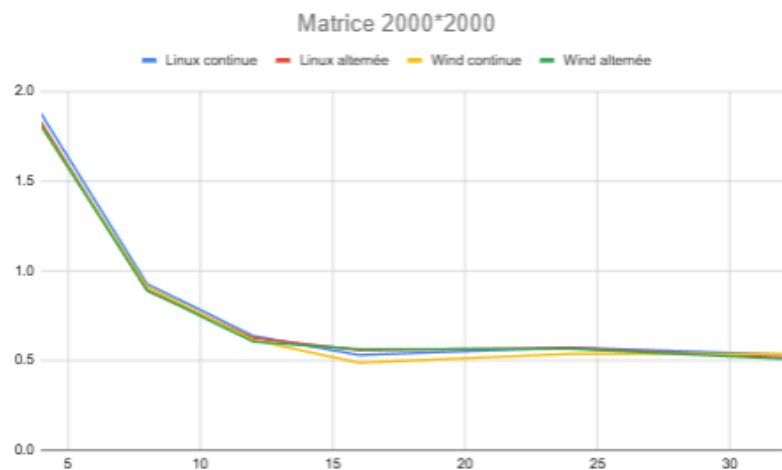


Matrice 1500*1500



Commentary

In this task, I applied the third and final optimization to my multithreaded matrix computation by aligning the data in memory on 64byte boundaries using `_mm_malloc`. This optimization ensures that each matrix starts exactly at the beginning of a CPU cache line, allowing the processor to load data in a single operation instead of two when data would otherwise straddle two cache lines.



Comparing these results with my TP4 optimization 2, the improvement is present but more modest than in the previous optimizations, which is exactly what was expected. For a 2000×2000 matrix on Linux with 16 threads, I went from 0.461s (optim 2) to 0.559s. I notice that some values fluctuate slightly, which is normal since the gain from memory alignment is less spectacular than the two previous optimizations and can be masked by the natural variability of measurements.

The optimal number of threads remains $T=16$ for all matrix sizes and both distribution modes, consistent with all my observations throughout this lab. Beyond 16 threads, context switching takes over and degrades performance, exactly as predicted by the course with the values $[0.25T, 0.5T, 0.75T, T, 1.5T, 2T]$.

Comparing the two distribution modes, alternating and contiguous continue to show very similar performance across all configurations, once again confirming that the row distribution strategy is not the determining factor for performance in this context.

Linux maintains its superiority over Windows for all tested configurations. For a 2000×2000 matrix with 16 threads, I obtain 0.559s on Linux versus 0.563s on Windows, a practically negligible gap, suggesting that 64-byte memory alignment benefits both environments equally, thereby reducing the OS-related differences observed in previous tasks.

CPU usage remains high and proportional to the number of threads up to $T=16$, reaching over 1300% on Linux for large matrices. Memory usage increases very slightly compared to previous versions due to the alignment potentially introducing small padding between allocations but remains negligible and predictable at N^2 .

To summarize all the optimizations applied throughout this lab, the results confirm that optimization 1 (transposed BT) is by far the most impactful, as it directly addresses the main bottleneck which is the cache miss. Optimizations 2 and 3 bring additional but more marginal gains, and their actual benefit heavily depends on the compiler's ability to generate optimal AVX instructions with the `-O2 -mavx2` flags.

Task 5: Cuda on GPU

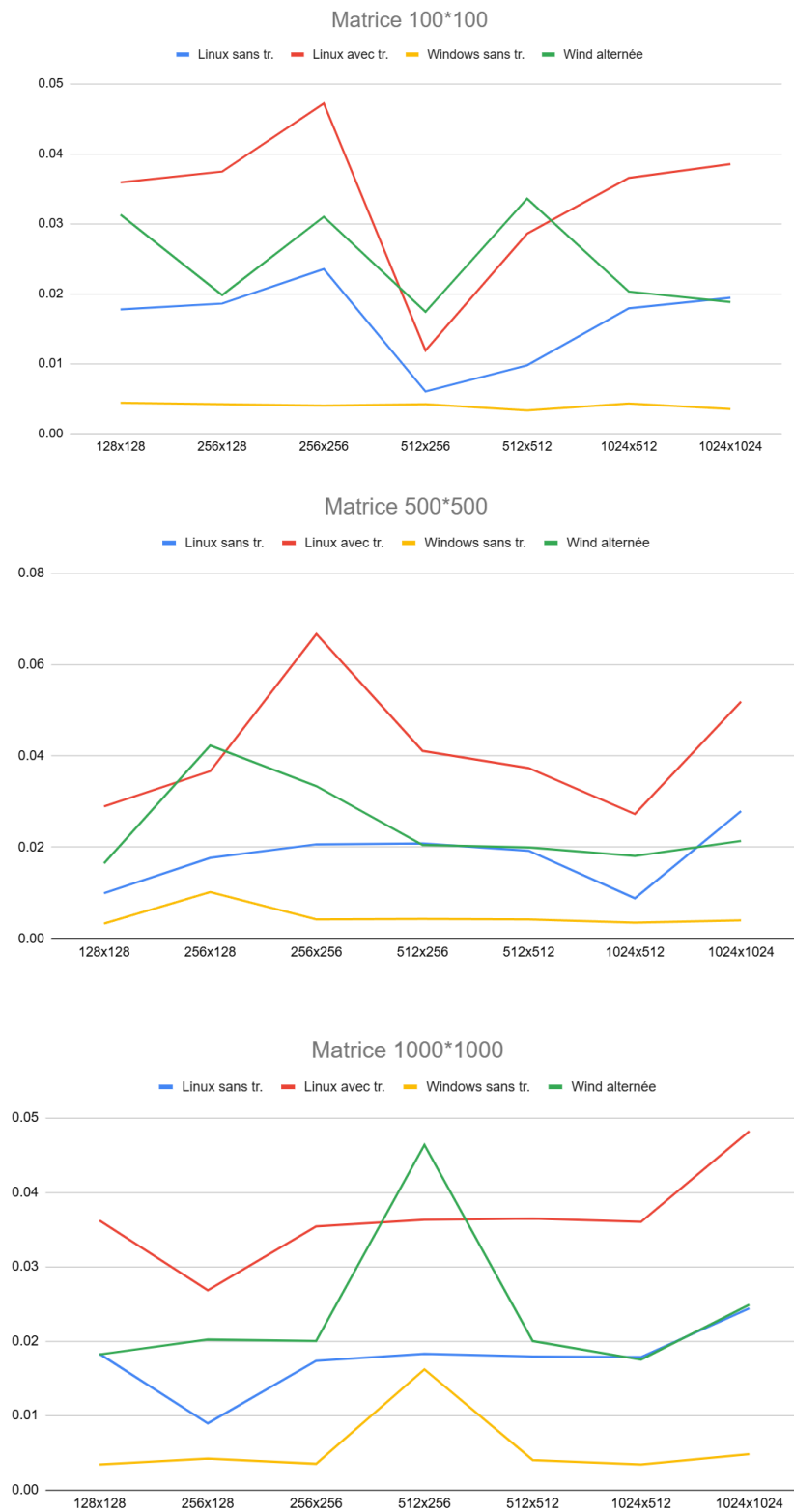
Value table under debian linux

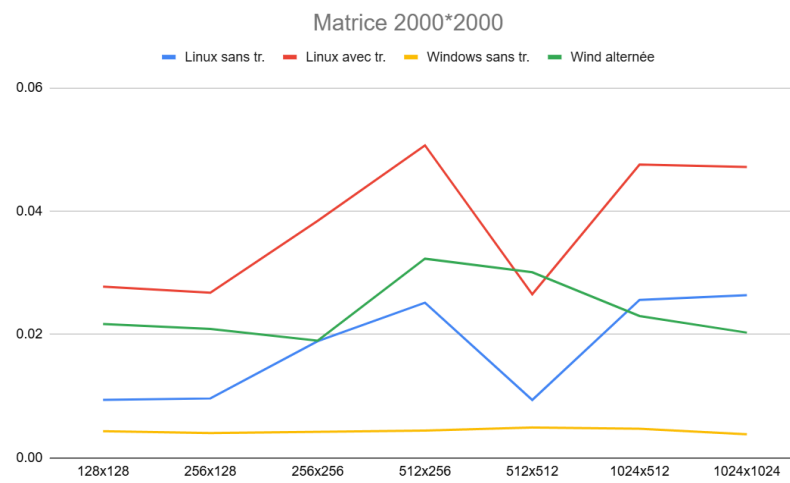
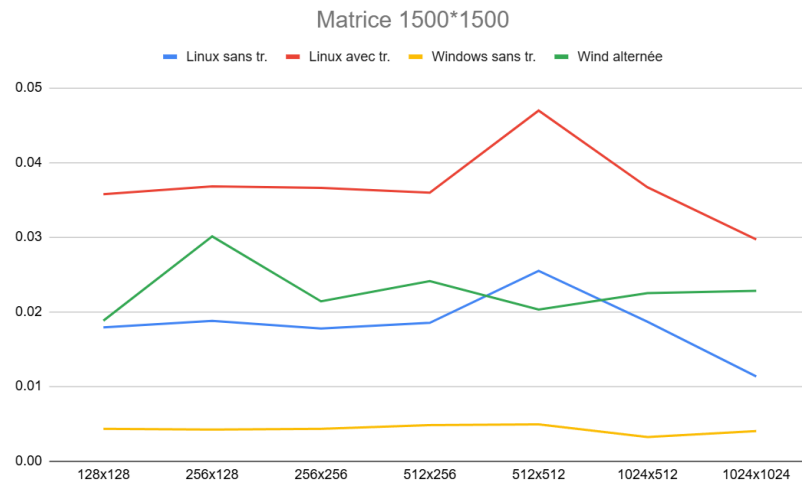
Ordi tour linux				
Matrix size	Number of blocs	Number of threads	Time [ms] without transfert	Time [ms] with transfert
100×100	128	128	0.017745	0.035924
	256	128	0.018584	0.037463
	256	256	0.023523	0.047207
	512	256	0.006012	0.011887
	512	512	0.009754	0.028572
	1024	512	0.017913	0.036557
	1024	1024	0.019412	0.038525
500×500	128	128	0.009961	0.028949
	256	128	0.017683	0.036678
	256	256	0.020661	0.066732
	512	256	0.020839	0.041109
	512	512	0.019261	0.037364
	1024	512	0.008828	0.027284
	1024	1024	0.027939	0.051911
1000×1000	128	128	0.018277	0.036228
	256	128	0.008934	0.026826
	256	256	0.017341	0.035428
	512	256	0.018277	0.036321
	512	512	0.017939	0.036466
	1024	512	0.017852	0.036036
	1024	1024	0.024401	0.048249
1500×1500	128	128	0.017909	0.035756
	256	128	0.018777	0.036811
	256	256	0.017746	0.036598
	512	256	0.01851	0.035959
	512	512	0.025475	0.046977
	1024	512	0.018659	0.036689
	1024	1024	0.011328	0.029684
2000×2000	128	128	0.009385	0.02776
	256	128	0.009617	0.026774
	256	256	0.018904	0.038435
	512	256	0.025167	0.050703
	512	512	0.00937	0.026532
	1024	512	0.025623	0.047589
	1024	1024	0.026385	0.047195

Value table under Windows 11

Ordi Tour Windows				
Matrix size	Number of blocs	Number of threads	Time [ms] without transfert	Time [ms] with transfert
100×100	128	128	0.0044	0.0313
	256	128	0.0042	0.0198
	256	256	0.004	0.031
	512	256	0.0042	0.0174
	512	512	0.0033	0.0336
	1024	512	0.0043	0.0203
	1024	1024	0.0035	0.0188
500×500	128	128	0.0033	0.0165
	256	128	0.0102	0.0423
	256	256	0.0042	0.0334
	512	256	0.0043	0.0205
	512	512	0.0042	0.02
	1024	512	0.0035	0.0181
	1024	1024	0.004	0.0214
1000×1000	128	128	0.0034	0.0182
	256	128	0.0042	0.0202
	256	256	0.0035	0.02
	512	256	0.0162	0.0464
	512	512	0.004	0.02
	1024	512	0.0034	0.0175
	1024	1024	0.0048	0.0249
1500×1500	128	128	0.0043	0.0188
	256	128	0.0042	0.0301
	256	256	0.0043	0.0214
	512	256	0.0048	0.0241
	512	512	0.0049	0.0203
	1024	512	0.0032	0.0225
	1024	1024	0.004	0.0228
2000×2000	128	128	0.0043	0.0217
	256	128	0.004	0.0209
	256	256	0.0042	0.019
	512	256	0.0044	0.0323
	512	512	0.0049	0.0301
	1024	512	0.0047	0.023
	1024	1024	0.0038	0.0203

Graphical Representation of Data





Commentary

Throughout this lab, I was able to concretely discover the power of GPU parallel computing with CUDA. Revisiting the work done in Matrix Lab 2 but this time executing it on the GPU allowed me to clearly grasp the difference between a classic sequential approach and a massively parallel one.

What struck me the most was the enormous impact of memory transfers between the CPU and the GPU. By comparing my measurements with and without transfers, I realized that these transfers often represent the majority of the total execution time, sometimes 2 to 5 times longer than the computation itself. This made me understand that parallelizing on GPU only truly makes sense when the volume of computation is large enough to amortize this transfer cost. For small matrices, almost all the benefit of parallelization is lost.

I also tested several block and thread configurations, starting from 128×128 as suggested, then exploring 256, 512 and 1024. I found that the optimal configuration is not universal: it depends on the matrix size. For large matrices (1500×1500 and 2000×2000), increasing the number of threads per block proved beneficial, especially when anticipating that a single thread may need to compute multiple elements when the matrix size exceeds the total number of available threads.

The comparison between Linux and Windows on the same machine also surprised me. I expected relatively similar results, but Windows turned out to be generally faster, particularly for computations without transfers. I think this mainly comes from differences in GPU drivers and memory management between the two systems.

Finally, this lab taught me the importance of properly calling `cudaDeviceSynchronize()` before measuring execution times. Without it, I would have obtained completely misleading measurements due to the asynchronous nature of CPU-GPU execution, a detail that seems minor but changes everything.

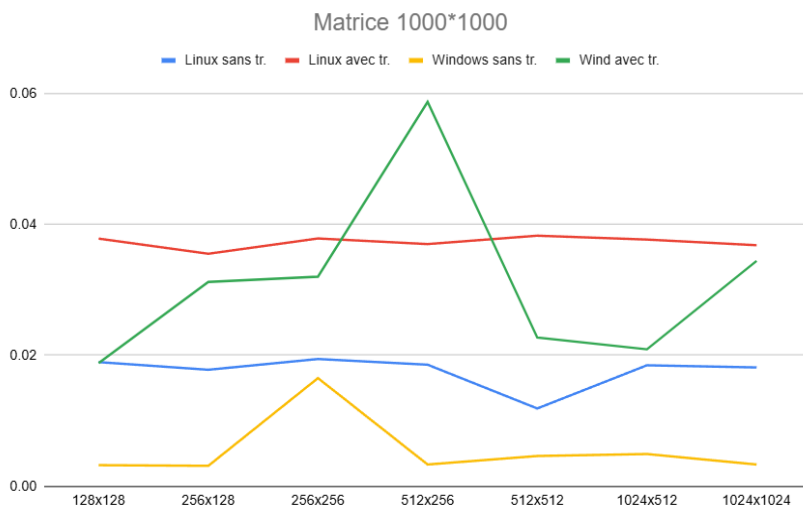
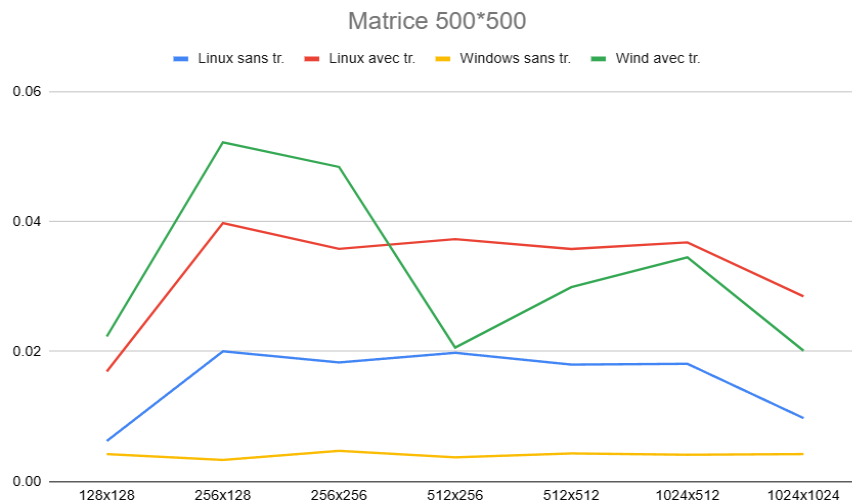
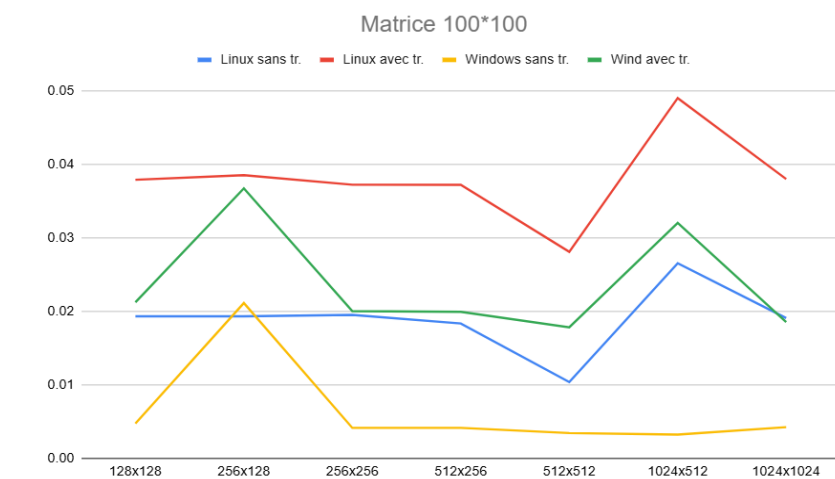
In the end, what I take away most from this lab is that programming on GPU is not just about "making code faster": it is about rethinking the way data and computations are structured to best exploit parallelism, while always keeping in mind the cost of memory transfers.

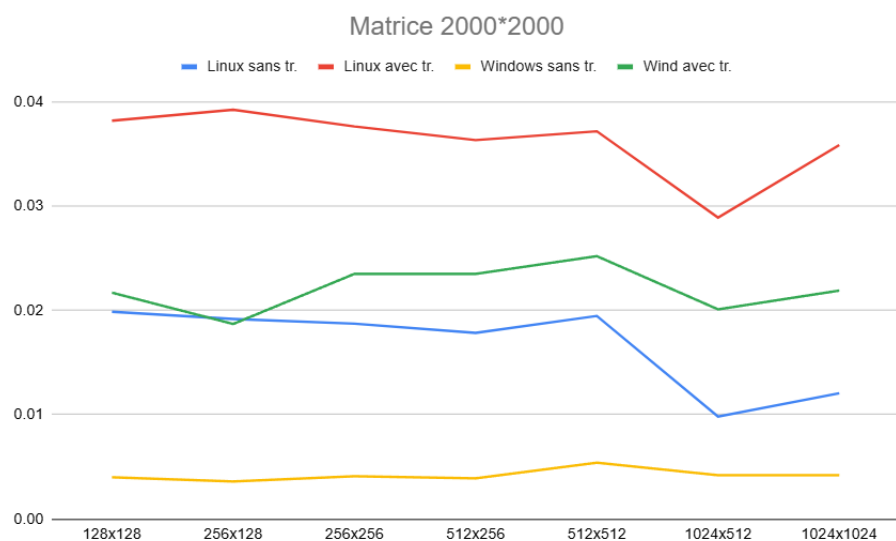
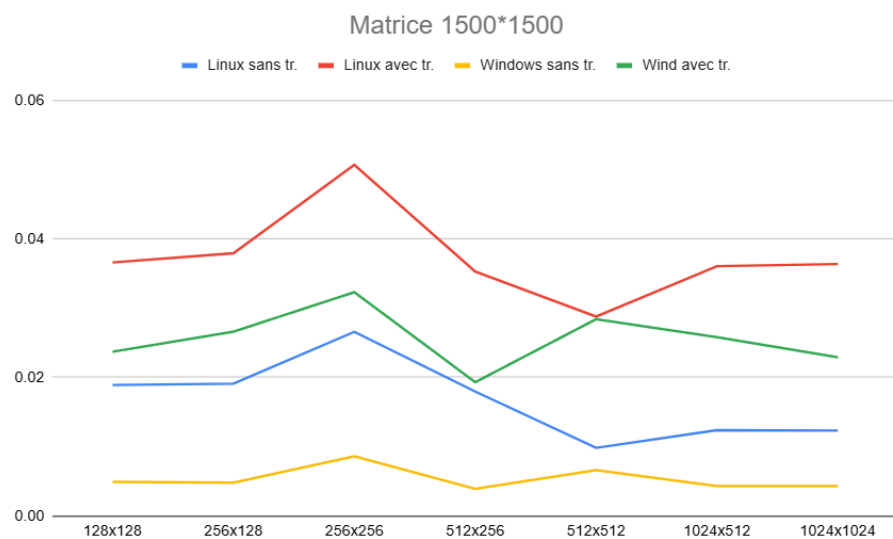
Task 5.1: CUDA on GPU with the Transpose Optimization

Value table

Ordi tour linux					Ordi Tour Windows				
Matrix size	Number of blocs	Number of threads	Time [ms] without transfert	Time [ms] with transfert	Matrix size	Number of blocs	Number of threads	Time [ms] without transfert	Time [ms] with transfert
100×100	128	128	0.019296	0.037868	100×100	128	128	0.0047	0.0212
	256	128	0.019292	0.038489		256	128	0.0211	0.0367
	256	256	0.019474	0.037201		256	256	0.0041	0.02
	512	256	0.018317	0.037183		512	256	0.0041	0.0199
	512	512	0.010351	0.028058		512	512	0.0034	0.0178
	1024	512	0.026517	0.048993		1024	512	0.0032	0.032
	1024	1024	0.019043	0.037971		1024	1024	0.0042	0.0185
500×500	128	128	0.006213	0.016898	500×500	128	128	0.0042	0.0223
	256	128	0.020019	0.039764		256	128	0.0033	0.0522
	256	256	0.018308	0.035795		256	256	0.0047	0.0484
	512	256	0.019789	0.037286		512	256	0.0037	0.0206
	512	512	0.017978	0.035779		512	512	0.0043	0.0299
	1024	512	0.018098	0.036781		1024	512	0.0041	0.0345
	1024	1024	0.009739	0.028482		1024	1024	0.0042	0.0201
1000×1000	128	128	0.018945	0.037812	1000×1000	128	128	0.0032	0.0188
	256	128	0.017774	0.035499		256	128	0.0031	0.0312
	256	256	0.019417	0.037833		256	256	0.0165	0.032
	512	256	0.018546	0.036976		512	256	0.0033	0.0587
	512	512	0.01186	0.038258		512	512	0.0046	0.0227
	1024	512	0.018458	0.037666		1024	512	0.0049	0.0209
	1024	1024	0.01813	0.036803		1024	1024	0.0033	0.0344
1500×1500	128	128	0.018896	0.0366	1500×1500	128	128	0.0049	0.0237
	256	128	0.019093	0.037924		256	128	0.0048	0.0266
	256	256	0.026571	0.050698		256	256	0.0086	0.0323
	512	256	0.017954	0.03527		512	256	0.0039	0.0193
	512	512	0.009816	0.028769		512	512	0.0066	0.0284
	1024	512	0.012381	0.036059		1024	512	0.0043	0.0258
	1024	1024	0.012321	0.036362		1024	1024	0.0043	0.0229
2000×2000	128	128	0.019873	0.038182	2000×2000	128	128	0.004	0.0217
	256	128	0.019179	0.039245		256	128	0.0036	0.0187
	256	256	0.018728	0.03764		256	256	0.0041	0.0235
	512	256	0.017845	0.036327		512	256	0.0039	0.0235
	512	512	0.019466	0.037174		512	512	0.0054	0.0252
	1024	512	0.009809	0.028901		1024	512	0.0042	0.0201
	1024	1024	0.012052	0.035855		1024	1024	0.0042	0.0219

Graphical Representation of Data





Commentary

With this Bonus Lab, I wanted to push the optimization of GPU matrix computation a step further by using the transpose of matrix B. The idea was simple on paper: instead of iterating through the columns of B, which implies non-contiguous memory accesses, I work exclusively with its rows by going through B^T . But seeing the concrete impact this has on performance was really interesting.

What struck me when comparing my results with those of Matrix Lab 5 is that the optimization becomes truly significant for large matrices. On Linux for example, I went from ~ 0.025 ms to approximately ~ 0.010 ms for a 2000×2000 matrix without transfers, representing a speedup close to $2.5\times$. For 1500×1500 and 500×500 matrices, I observe the same kind of improvement. However, for small matrices (100×100), the gain is almost nonexistent, or even slightly negative. This is easy to explain: the additional cost of creating B^T , which is included in the measurement as required is not amortized when the computation is already extremely fast.

I also understood why this optimization works so well: by working row by row on both matrices, memory accesses become contiguous, which allows better exploitation of the cache and memory access coalescing on the GPU. This was a concept that felt abstract to me at the beginning of the course, but seeing concrete numbers confirm the theory really helped me internalize this aspect.

On Windows, the effect of the optimization is less spectacular since computation times were already very low in the previous lab. This clearly shows that the effectiveness of an optimization always depends on the context: hardware configuration, data size, and system memory management.

Regarding memory transfers, the observation remains the same as in the previous lab: they continue to dominate the total execution time by a large margin. Even with an efficient computation optimization, there is no escaping the cost of CPU $\leftrightarrow$ GPU communication. This confirmed to me that optimizing the computation alone is not enough: one must also think about how data flows between the host and the device.

In the end, this Bonus Lab made me realize that GPU performance cannot be reduced to raw parallelism. The way data is organized in memory often has just as much, if not more, impact than the number of launched threads. This is a reflex I will keep going forward: before looking to parallelize further, it is always worth asking whether the memory access pattern is efficient first.

General Conclusion

This series of tasks provided a comprehensive exploration of different approaches to matrix multiplication, ranging from basic sequential processing to advanced GPU acceleration using CUDA. Each method offered valuable insights into performance, scalability, memory access patterns, and system resource utilization, and together they paint a clear picture of how hardware and software choices interact to determine real-world performance.

-Task 1 (Sequential) highlighted the fundamental limitations of single-threaded execution. While straightforward to implement, it quickly became inefficient for large matrices. However, the three optimizations applied throughout this task introducing the transposed matrix BT for cache-friendly memory access, converting to float to leverage AVX/FMAD instructions, and aligning data on 64-byte boundaries demonstrated that even sequential code can be dramatically improved by thinking carefully about how data is accessed and processed. The combination of all three optimizations delivered a speedup of up to 10x compared to the naive version, which was honestly one of the most surprising results of the entire lab.

Task 2 (One Thread per Element) demonstrated that excessive parallelism can be counterproductive. Creating a thread for every matrix element introduced massive overhead and instability, leading to crashes beyond small matrix sizes. This approach quickly proved impractical and reinforced the idea that more threads does not automatically mean better performance.

Task 3 (One Thread per Row) struck a more balanced approach, achieving significant performance gains while maintaining system stability. By assigning one thread per row rather than per element, the overhead was drastically reduced while still exploiting the available CPU cores effectively.

Task 4 (Dynamic Thread Allocation) offered the best CPU-side performance by adapting the number of threads to the available hardware resources. Testing values across [0.25T, 0.5T, 0.75T, T, 1.5T, 2T] consistently confirmed T=16 as the optimal configuration on my machine, beyond which context switching overhead begins to cancel out the benefits of additional parallelism. The three optimizations applied here BT transposition, float/AVX conversion, and 64-byte alignment further confirmed that memory access efficiency is often more impactful than raw thread count.

Task 5 (CUDA on GPU) clearly outperformed all CPU-based methods for large matrices. By leveraging the GPU's massively parallel architecture, computation times dropped to fractions of a millisecond. However, this task also revealed an important limitation: memory transfers between the CPU and GPU can represent 2 to 5 times the actual computation time, meaning GPU acceleration only truly pays off when the workload is large enough to amortize this cost. The Bonus Task further reinforced this by showing that applying the BT transposition on the GPU improving memory coalescing delivered an additional speedup of up to 2.5x for large matrices, while having negligible or even slightly negative impact on small ones.

Overall, this project illustrates that performance optimization is never a one-size-fits-all solution. Each layer of optimization whether algorithmic, memory-related, or hardware-specific contributes differently depending on the problem size and execution context. If I had to retain one key lesson from this entire lab, it would be this: before adding more parallelism, always make sure the memory access pattern is efficient. That single principle explained the most impactful results I observed, both on CPU and GPU.